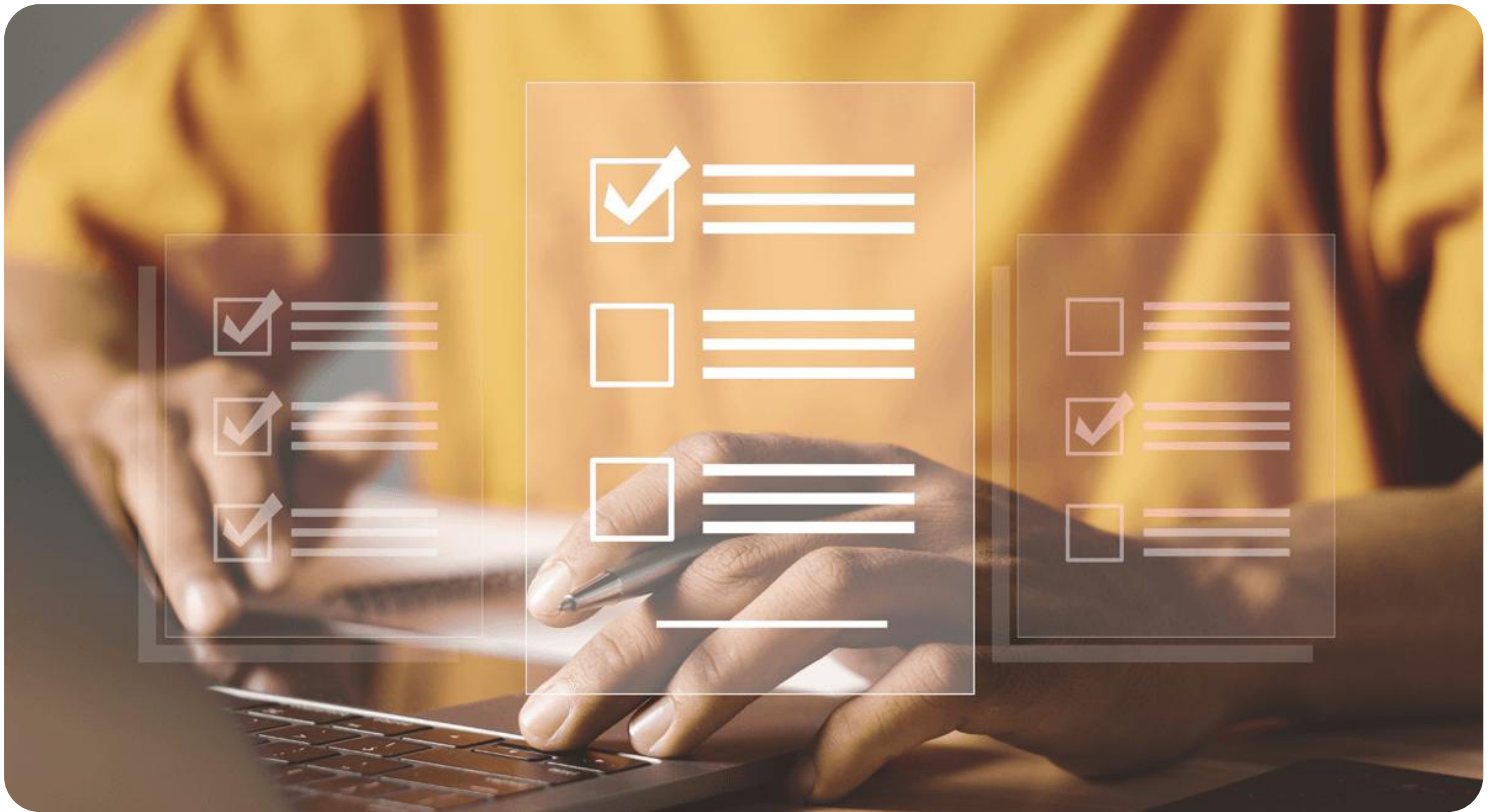




Semana 3

Desarrollo Backend II (PBY2202)

Formato de respuesta

Nombre estudiante: Raúl Zúñiga – Francisco Henríquez	
Asignatura: Desarrollo Backend II	Carrera: Desarrollo de Aplicaciones
Profesor: Jorge Carmona	Fecha: 06-06-2026

Descripción de la actividad

En esta tercera semana, realizarás una actividad sumativa grupal (2 a 3 integrantes), llamada "Integrando seguridad en aplicaciones Backend", donde a partir de una aplicación backend previamente desarrollada, analices, configures, desarrolles e integres mecanismos de seguridad utilizando frameworks y herramientas como Spring Security, JWT, LDAPS e IDaaS. Además, tendrás que asegurar la protección de los componentes backend frente a amenazas conocidas y garantizar la interoperabilidad, siguiendo buenas prácticas y estándares de la industria y de acuerdo con requerimientos específicos.

Instrucciones específicas

Para el desarrollo de la actividad sumativa de la semana, recordemos el caso visto durante toda la experiencia, que requiere la implementación de microservicios para mejorar su arquitectura de software:

Contexto

"MiniMarket Plus" es una cadena de minimarkets en expansión, fundada en la ciudad de Santiago en el año 2005. Con más de 18 años de presencia en el mercado, ha logrado establecerse y actualmente cuenta con 10 sucursales distribuidas en la Región Metropolitana, con planes de expansión a otras regiones del país.

En ella, los clientes pueden encontrar una amplia variedad de productos de consumo diario:

- **Abarrotes:** alimentos no perecibles, conservas, productos enlatados, arroz, legumbres, etc.
- **Bebidas:** refrescos, jugos, aguas minerales, cervezas y licores.
- **Lácteos y congelados:** leche, yogur, quesos, mantequilla, helados y otros productos que requieren refrigeración.
- **Artículos de limpieza:** detergentes, desinfectantes, utensilios de limpieza y productos para el cuidado del hogar.
- **Cuidado personal:** artículos de higiene personal, cosméticos y productos de farmacia básica.

Necesidades específicas:

- Autenticación Segura:

- Implementar un sistema de autenticación robusto que permita a los usuarios iniciar sesión de manera segura.
- Utilizar JWT para gestionar las sesiones de los usuarios en un entorno sin estado (stateless).
- **Autorización Basada en Roles:**
 - Definir y gestionar diferentes roles de usuario (clientes, empleados, administradores).
 - Controlar el acceso a los recursos y operaciones del backend según los roles y permisos asignados.

Necesidades tecnológicas:

Con el objetivo de modernizar sus operaciones y fortalecer su sistema de seguridad, “MiniMarket Plus” requiere incorporar una arquitectura backend que permita:

Gestión de inventario centralizado:

- Controlar el stock de productos en tiempo real en todas las sucursales.
- Automatizar pedidos a proveedores cuando el stock esté por debajo de cierto umbral.
- Generar reportes de productos más vendidos y tendencias de compra.
- **Sistema de ventas y pedidos en línea:**
 - Permitir a los clientes consultar la disponibilidad de productos en cada sucursal.
 - Realizar reservas de despacho para recoger en sucursal o solicitar envío a domicilio.
- **Gestión de usuarios y fidelización:**
 - Registro y autenticación de usuarios, con perfiles de clientes y empleados.
 - Implementar un sistema de puntos por compras realizadas.
 - Enviar notificaciones sobre promociones.

- **Seguridad y protección de datos:**

- Proteger la información personal y transacciones de los clientes.
- Garantizar el cumplimiento de la Ley de Protección de Datos Personales en Chile.

Implementar mecanismos de autenticación y autorización robustos.



Importante

El código del backend de "MINIMARKET PLUS" será proporcionado para su descarga a través del Aula Virtual. Deberás utilizar este backend como base para la actividad, enfocándote en el análisis y configuración del framework de seguridad según los requerimientos proporcionados.

Preguntas de apoyo

Para apoyarte en tu análisis de proyecto, guíate por las siguientes preguntas:

- ¿Cuáles son las principales amenazas de seguridad que enfrenta el backend reutilizado del proyecto desarrollado en "Desarrollo Backend I" y cómo pueden impactar su funcionalidad y objetivos?
- ¿Qué estrategias de seguridad son más apropiadas para mitigar estas amenazas en el contexto de una aplicación backend?
- ¿Qué framework de seguridad se adapta mejor a los requerimientos del proyecto y por qué?
- ¿Cómo implementaste la autenticación y autorización para los diferentes roles en el framework seleccionado utilizando Spring Security?
- ¿Qué medidas adicionales implementaste para asegurar el cumplimiento de normativas de protección de datos aplicables?

Ahora que has revisado y analizado el caso, tendrás que realizar los siguientes pasos:

Paso 1: Análisis de estrategias de implementación de frameworks de seguridad

- Revisión de requerimientos del cliente y análisis de amenazas:
 - Identifica las principales amenazas en "MINIMARKET PLUS":
 - Acceso no autorizado a datos sensibles.
 - Ataques de inyección SQL, XSS, y CSRF.
- Definición de los requerimientos de seguridad:
 - Implementa autenticación y autorización seguras para usuarios y empleados.
 - Configura un monitoreo básico de actividades sospechosas.
- Identificación de usuarios y roles:
 - Define los roles: cliente, empleado, administradores.
 - Establece niveles de seguridad diferenciados según el rol.
- Exploración de estrategias de autenticación y selección de la más apropiada
 - Revisa las siguientes opciones:
 - Autenticación en memoria (para pruebas rápidas).
 - Autenticación basada en base de datos (JDBC).
 - Integración con LDAP.
 - Autenticación con JWT (seleccionada por requerimientos del cliente).
 - Justifica la elección de JWT basado en las necesidades de seguridad y escalabilidad.

Paso 2: Configuración del framework de seguridad

- Preparación del proyecto:
 - Descarga el código proporcionado desde el Aula Virtual.
- Configura el entorno de desarrollo con las dependencias necesarias:
 - spring-boot-starter-security
 - spring-boot-starter-web
 - jjwt
- Implementación de seguridad:
 - Autenticación:
 - Implementa una clase de servicio (UserDetailsService) para cargar los detalles del usuario desde la base de datos.
 - Asegura las contraseñas usando BCrypt.
 - JWT:
 - Configura un JwtUtil para generar y validar tokens.
 - Implementa un filtro que valide los tokens antes de procesar las solicitudes.
 - Autorización Basada en Roles:
 - Define las reglas de acceso usando anotaciones como @PreAuthorize.
 - Configura los endpoints para restringir el acceso según los roles.
 - Pruebas iniciales:
 - Realiza pruebas funcionales para verificar que cada rol tenga acceso únicamente a los recursos permitidos.

Paso 3: Documentación del proceso

Elabora un informe que incluya:

1. Análisis del caso: amenazas identificadas y justificación de la estrategia seleccionada.
2. Guía de configuración: paso a paso de la configuración del framework de seguridad.
3. Explicación técnica: cómo las configuraciones realizadas protegen el backend contra amenazas específicas.

Entregables

En este apartado deberás adjuntar lo solicitado en los pasos anteriores sobre:

1. Análisis y selección de la estrategia y framework de seguridad.

Para poder dar cumplimiento a las necesidades del cliente de esta semana se revisaron las necesidades del cliente y las principales amenazas que enfrenta el proyecto MINIMARKET PLUS.

- **Acceso no autorizado a datos sensibles**

Una de las amenazas más importantes es que los usuarios no autenticados o usuarios con permisos insuficientes accedan a recursos internos del backend. Por ejemplo, un cliente debería administrar usuarios, modificar inventarios ni consultar información sensible de venta. Ya que esto podría tener impactos como exposición de información personal, modificación indebida de usuarios o roles, alteración del inventario y acceso a operaciones administrativas.

- **Robo o exposición de credenciales.**

Si las contraseñas se almacenan en texto plano o se devuelven en respuesta JSON, un ataque podría reutilizarlas para ingresar al sistema, pudiendo generar suplantación de identidad, escalamiento de privilegios y pérdida de confianza de nuestro cliente y sus empleados en el sistema.

- **Manipulación de sesiones**

En una API REST, mantener sesiones tradicionales en el servidor puede generar problemas de escalabilidad y dependencias del estado interno de cada instancia. Además, si una sesión se gestiona de forma insegura, podría ser reutilizada por terceros. Esto podría generar un uso indebido de sesiones activas, mayor dificultad para escalar el backend y dependencia de almacenamiento de sesiones del lado del servidor.

- **Ataques de inyección SQL**

La inyección SQL ocurre cuando entradas del usuario se conectan directamente en consultas. En este proyecto, el uso de Spring Data JPA ayuda a mitigar este riesgo porque las consultas de se gestionan mediante repositorios y parámetros controlados, si no se hace de esta manera, podríamos tener Lecturas no autorizada de datos, modificación o eliminación de riesgos y compromiso de información sensible.

- **Ataques de inyección XSS**

Aunque el backend no renderiza vistas HTML principales, pude recibir y devolver datos que luego sean mostrados por un frontend. Si no se validan o limpian correctamente, esto podría generar un riesgo de inyección de scripts en una capa del cliente. Esto podría generar Robo de tokens en el frontend. Manipulación de interfaces y suplantación de acciones del usuario.

- **Ataque de CSRF**

Suele afectar las aplicaciones que usan cookies de sesión automáticamente enviadas por el navegador. En este proyecto se trabaja con JWT enviado en el encabezado “Authorization”, por lo que el riesgo es menor que en una aplicación web tradicional con sesiones. Ahora como impactos que se podrían generar si se usaran cookies sin protección, serían la ejecución de acciones no autorizadas y cambios realizados sin consentimiento del usuario autenticado.

- **Definición de los requerimientos de seguridad:**

A partir del caso de la semana pasada y considerando las amanezas identificadas en los puntos anteriores, de establecieron los siguientes requerimientos:

- A- Autenticación mediante usuario y contraseña.
- B- Cifrado de contraseña con BCrypt.
- C- Emisión de tokens JWT firmados.
- D- Validación del token en cada solicitud protegida.
- E- Arquitectura stateless sin sesiones de servidor.
- F- Autorización basada en roles.
- G- Roles definidos: “Cliente”, “Empleado” y “Administrador”.
- H- Restricción de endpoints según rol
- I- Uso de “@PreAuthorize” para reforzar reglas de autorización a nivel de método.
- J- Registro básico de evento de login exitoso y fallido.

- **Estrategias de autenticación y selección de las más apropiadas.**

- Autenticación en memoria (para pruebas rápidas:

Esta alternativa permite definir usuarios directamente en la configuración de Spring Security. Es útil para pruebas rápidas o ejemplos académicos iniciales, pero no es adecuada para MINIMARKET PLUS porque los usuarios deben persistirse y gestionarse desde una base de datos en el futuro.

Ventaja: Es de fácil configuración y útil para pruebas simples.

Desventajas: No permite gestión real de usuarios, no escala para clientes, empleados y administradores y no se adapta bien a un sistema productivo.

- Autenticación basada en base de datos (JDBC).

Esta estrategia permite cargar usuarios desde una base de datos mediante un servicio como “UserDetailsService”. Es adecuada para el proyecto porque los usuarios y roles forman parte del modelo del sistema.

Ventajas: Permite persistir usuarios y roles, además que facilita la administración del sistema y se integra bien con JPA y Spring Security.

Desventajas: Requiere implementar repositorios, entidades y servicios de carga de usuarios, además debe asegurar que la contraseña este guardada correctamente.

- Integración con LDAP.

De acuerdo con lo que pasamos en la última semana, comprendemos que LDAP es una herramienta útil en organizaciones que ya cuentan con directorios corporativos centralizados. En el caso de MINIMARKET PLUS, no se especifica una infraestructura LDAP existente, por lo que sería una solución más compleja de lo necesario para esta etapa

Ventajas: Centraliza identidades empresariales y es adecuado para organizaciones grandes con directorios internos.

Desventajas: Posee una mayor complejidad de configuración y no responde directamente al requerimiento de JWT del caso.

- Autenticación con JWT (seleccionada por requerimientos del cliente).

Se seleccionó la opción de JWT basado en las necesidades de seguridad y escalabilidad, ya que permite manejar autenticación en una API REST stateless. El usuario inicia sesión, recibe un token firmado y luego envía ese token en cada solicitud mediante el encabezado de "Authorization: Bearer"

Ventajas: Este método es compatible con arquitectura REST y microservicios, no requiere guardar sesiones en el servidor, no permite escalar horizontalmente con menor acoplamiento, el token puede incluir datos útiles como user name y roles, además se puede validar firma y expiración en cada request.

Desventajas: Requiere proteger correctamente la clave secreta, además que si un token es robado puede usarse hasta que expire, también debe manejarse cuidadosamente en el frontend para evitar exposiciones.

Análisis del Framework de seguridad seleccionado

El framework seleccionado es Spring Security, integrado con JWT mediante la librería JJWT. Se seleccionó Spring Security porque es el framework de seguridad estándar dentro del ecosistema Spring Boot. Permite configurar autenticación, autorización, filtros, cifrado de contraseña, políticas de sesión y protección de endpoints de forma robusta y extensible.

En nuestro proyecto complementamos lo anterior con lo siguiente:

IMPLEMENTACIÓN	QUE HACE
spring-boot-starter-security	Configuración base de seguridad
spring-boot-starter-web	Exposición de endpoints REST
jjwt-api, jjwt-impl y jjwt-jackson	Generación y validación de tokens JWT

spring-security-test	Pruebas de seguridad con MocMvc
----------------------	---------------------------------

Justificación de Spring Security y JWT

La justificación de la utilización de éstas tecnologías es debido a que la combinación de ambas es apropiada para los requisitos del proyecto por las siguientes razones:

- El backend expone servicios REST, por lo que una estrategia stateless es más adecuada para sesiones radicionales.
- JWT permite validar cada solicitud sin almacenar sesiones en memoria del servidor.
- Spring Security permite restringir endpoints según roles de negocio.
- BCrypt permite almacenar contraseñas de forma segura.
- "UserDetailsService" permite cargar usuarios reales desde la base de datos.
- @PreAuthorize refuerza la autorización directamente en los controladores.
- La solución puede crecer hacia una arquitectura de microservicios, ya que otros servicios podrían validar tokens sin depender de una sesión central.

Modelos de roles seleccionados

El proyecto de esta semana modifica el rol de Gerente a administrador y los demás roles se mantienen iguales. El proyecto define tres niveles principales de acceso:

ROLES	ACCESO
Cliente	usuario que puede autenticarse, consultar catalogo y utilizar funcionalidades asociadas al carrito.
Empleado	usuario interno que puede operar ventas, inventario y gestionar recursos comerciales.
Administrador	usuario con mayor privilegio, encargado de administrar usuarios y acceder a operaciones internas criticas.

Esta separación aplica el principio de mínimo privilegio, ya que cada usuario accede solo a las funcionalidades necesarias para su rol

Relación entre amenazas y controles implementados

Amenaza	Control implementado
Acceso no autorizado	JWT, Spring Security, reglas por endpoints y @PreAuthorize
Robo de credenciales	Cifrado de contraseñas con BCrypt y ocultamiento de password en respuesta JSON
Manipulación de token	Firma del JWT con clave secreta y validación de expiración
Uso de sesiones inseguras	Configuración stateless con SessionCreationPolicy.STATELESS
Inyección SQL	Uso de Spring Data JPA y repositorios
CSRF	Deshabilitado para API REST stateless que usa encabezado Authorization
Actividad sospechosa	Registro básico de login exitoso y fallido

La estrategia seleccionada para la actividad de esta semana de MINIMARKET PLUS es la implementación de seguridad con Spring Security, autenticación basada en base de datos (para el futuro), contraseñas cifradas con BCrypt, sesiones stateless mediante JWT y autorización basada en roles usando reglas de seguridad y anotaciones @PreAuthorize. Con esta selección se cumple los requerimientos de protección de endpoints sensibles, diferencia permisos entre clientes, empleados y administradores, evita sesiones del lado del servidor y permite una base escalable para arquitectura moderna del backend.

2. Guía de configuración y explicación de la implementación.

Esta guía contempla la configuración e implementación de seguridad aplicada al backend de la actividad de esta semana. El objetivo es poder explicar cómo se integró Spring Security con JWT para autenticar usuarios, proteger endpoints y controlar el acceso seguro de roles.

La implementación consideró tres roles principales:

- CLIENTE
- EMPLEADO
- ADMINISTRADOR

Cada rol tiene permisos diferenciados sobre los recursos del sistema, los cuales fueron especificados en el punto uno.

Configuración de dependencias

La configuración en el archivo la vamos a iniciar en el pom.xml, donde se incorporarán las dependencias para seguridad, API REST, JWT, persistencia y pruebas.

Como dependencias principales dejamos las siguientes:

Dependencias	Lo que hacen
spring-boot-starter-security	permite integrar Spring Security al proyecto
spring-boot-starter-web	permite exponer endpoints REST
spring-boot-starter-data-jpa	permite trabajar con entidades y repositorios
jjwt-api	contiene la API principal para trabajar con JWT
jjwt-impl	implementación interna de JWT
jjwt-jackson	soporte para serializar y deserializar claims del token
spring-security-test	permite probar seguridad con MockMvc

Estas dependencias permitirán autenticar usuarios, generar tokens, validar solicitudes y realizar pruebas automatizadas de acceso.

Imágenes de las dependencias antes descritas

```

<dependencies>
  <!--Dependencia de Spring JPA necesaria para trabajar con servicios de RESTful-->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <!--Dependencias de Spring Security-->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <!--Dependencias de Spring Web-->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!--Dependencias JWT para generar y validar tokens firmados-->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
  </dependency>
</dependencies>

```

```

  <!--Dependencias necesarios para poder realizar pruebas unitarias al código-->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

```

Propiedades de la configuración (application.properties)

En nuestro archivo application.properties, se van a configurar la base de datos H2, JPA y los parámetros necesarios para utilizar JWT.

Imagen de la implementación de application.properties

```
src > main > resources > application.properties
1  spring.application.name=minimarket
2  # Configuración de H2 en memoria
3  spring.datasource.url=jdbc:h2:mem:testdb
4  spring.datasource.driver-class-name=org.h2.Driver
5  spring.datasource.username=sa
6  spring.datasource.password=
7  spring.h2.console.enabled=true
8
9  # Configuración de JPA e Hibernate
10 spring.jpa.hibernate.ddl-auto=update
11 spring.jpa.show-sql=true
12 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
13
14 # Configuración JWT: secreto Base64 de al menos 256 bits y expiración de 1 hora
15 app.jwt.secret=VGhpcy1pcy1hLWR1bW8tc2VjcmV0LWtleS1mb3ItTWluaU1hcmtldC1QbHVzLTlwMjY=
16 app.jwt.expiration-ms=3600000
17
```

App.jwt.secret: corresponde a la clave usada para firmar los tokens JWT. En esta entrega se utiliza una clave de demostración codificada en Base64

App.jwt.expiration-ms: definimos la duración del token. En este caso lo dejamos con una expiración de una hora-

NOTA IMPORTANTE: Si la aplicación pasa a un entorno de producción la clave secreta no debería quedar escrita nunca directamente en el application.properties, debería cargarse desde variables de entorno o desde un gestor de secretos.

Modelo de usuarios y roles

La seguridad se apoya en dos entidades principales:

- Usuario
- Rol

La entidad Usuario va a contener:

- Id

- Username
- Password
- Roles

La propiedad password esta marcada con `JsonProperty.Access.WRITE_ONLY`, por lo que puede describirse en solicitudes, pero no se expone en respuestas JSON.

La entidad Rol contiene:

- Id
- Nombre
- Usuario

Los usuarios y roles se relacionan mediante una relación muchos a muchos. Esto permite que un usuario pueda tener uno o más roles asociados.

Imágenes de la Entidad Usuario

```
7  @Entity
8  public class Usuario {
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12
13     @Column(nullable = false, unique = true)
14     private String username;
15
16     @Column(nullable = false)
17     // La contraseña puede recibirse al crear/editar, pero no se expone en respuestas JSON.
18     @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
19     private String password;
20
21     @ManyToMany(fetch = FetchType.EAGER)
22     @JoinTable(
23         name = "usuario_roles",
24         joinColumns = @JoinColumn(name = "usuario_id"),
25         inverseJoinColumns = @JoinColumn(name = "rol_id")
26     )
27     private Set<Rol> roles;
28
29     // Getters y Setters
30     public Long getId() {
31         return id;
32     }
33
34     public void setId(Long id) {
35         this.id = id;
36     }
37
38     public String getUsername() {
39         return username;
40     }
41
42     public void setUsername(String username) {
43         this.username = username;
44     }
45
46     public String getPassword() {
47         return password;
48     }
```

```
49
50     public void setPassword(String password) {
51         this.password = password;
52     }
53
54     public Set<Rol> getRoles() {
55         return roles;
56     }
57
58     public void setRoles(Set<Rol> roles) {
59         this.roles = roles;
60     }
61 }
```

```

1  package com.minimarket.entity;
2
3  import com.fasterxml.jackson.annotation.JsonIgnore;
4  import jakarta.persistence.*;
5  import java.util.Set;
6
7  @Entity
8  public class Rol {
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12
13     @Column(nullable = false, unique = true)
14     private String nombre;
15
16     @ManyToMany(mappedBy = "roles")
17     @JsonIgnore
18     private Set<Usuario> usuarios;
19
20     // Getters y Setters
21     public Long getId() {
22         return id;
23     }
24
25     public void setId(Long id) {
26         this.id = id;
27     }
28
29     public String getNombre() {
30         return nombre;
31     }
32
33     public void setNombre(String nombre) {
34         this.nombre = nombre;
35     }
36
37     public Set<Usuario> getUsuarios() {
38         return usuarios;
39     }
40
41     public void setUsuarios(Set<Usuario> usuarios) {
42         this.usuarios = usuarios;
43     }
44 }
45

```

Inicialización de datos de prueba

La clase DataInitializer crea automáticamente los roles y usuarios iniciales cuando la bse de datos está vacía. Para nuestra aplicación, esta va a generar lo siguiente:

- CLIENTE

- EMPLEADO
- ADMINISTRADOR

Los usuarios de prueba se van a crear de la siguiente forma:

Usuarios	Contraseña
cliente	Cliente123
empleado	Empleado123
administrador	Administrador123

Las contraseñas se almacenan cifradas con BCrypt, no en texto plano.

Esta inicialización permite probar rápidamente el funcionamiento de autenticación y autorización sin cargar datos manualmente.

Imagen de nuestra Clase Dantainialrz

```

27  @Override
28  public void run(String... args) {
29      // Se crea roles base solicitados por la actividad si la BD parte vacia.
30      Rol cliente = obtenerOCrearRol(nombre: "CLIENTE");
31      Rol empleado = obtenerOCrearRol(nombre: "EMPLEADO");
32      Rol administrador = obtenerOCrearRol(nombre: "ADMINISTRADOR");
33
34      // Se usuarios de prueba con contraseñas BCrypt para validar los niveles de acceso.
35      crearUsuarioSiNoExiste(username: "cliente", password: "Cliente123", cliente);
36      crearUsuarioSiNoExiste(username: "empleado", password: "Empleado123", empleado);
37      crearUsuarioSiNoExiste(username: "administrador", password: "Administrador123", administrador);
38  }

```

Cifrado de las contraseñas utilizando BCrypts

Spring Security utiliza un PasswordEncoder configurado en SecurityConfig

Imagen de la clase SecurityConfig el cual tiene la utilización de PasswordEncoder

```

@Bean
public PasswordEncoder passwordEncoder() {
    // BCrypt protege las contraseñas almacenadas contra lectura directa.
    return new BCryptPasswordEncoder();
}

```

BCrypt se va a utilizar para transformar la contraseña original en un hash seguro antes de guardarla en la base de datos.

El cifrado se va a aplicar de la siguiente forma;

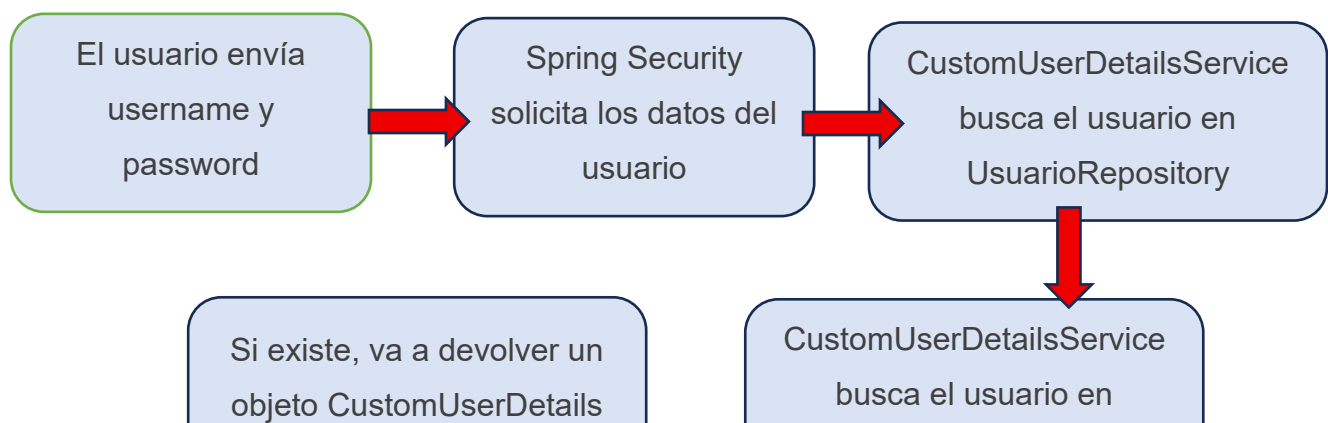
DanatInitializer.java	Crea usuarios iniciales
AuthController	Va a registrar nuevos clientes
UsuarioServiceImpl	Va a guardar o actualizar los usuarios

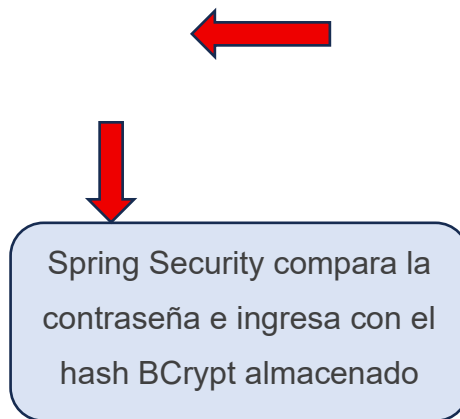
Esto va a proteger las credenciales en caso de que la base de datos sea expuesta.

Carga de usuarios desde la base de datos

La clase CustomUserDetailsService implementa UserDetailsService, que es el mecanismo usado por Spring Security para cargar datos del usuario durante el login

El flujo que presenta sería así





Este enfoque permite autenticar usuarios reales guardados en la base de datos.

NOTA: Recordemos que en esta actividad estamos trabajando con una base local en memoria H2, pero este flujo aplica para cualquier persistencia que utilicemos ya sea mediante SQL, POSTGRES SQL entre otros.

Adaptación de los usuarios a Spring Security

La clase CustomUserDetails adapta la entidad Usuario al formato que Spring Security necesita

Responsabilidades principales:

- Entrega el username
- Entrega la contraseña cifrada
- Convierte los roles del usuario en authorities
- Normaliza los roles con el prefijo ROLE_

Esto se refleja de la siguiente forma:

Ejemplo:

ADMINISTRADOR = ROLE_ADMINISTRADOR

CLIENTE = RILE_CLIENTE

EMPLEADO = ROLE_EMPLEADO

Esto es necesario porque métodos como como `hasRole("ADMINISTRADOR")` espera internamente una authority llamada `ROLE_ADMINISTRADOR`.

Generación y validación de JWT

La clase JwtUtil se encarga de crear y validar tokens JWT

Al generar un token, se incluyen:

- Subject: nombre del usuario
- Roles: roles del usuario autenticados
- issuedAt: fecha de emisión
- expiration: fecha de expiración
- Firma del HMAC usando la clave secreta

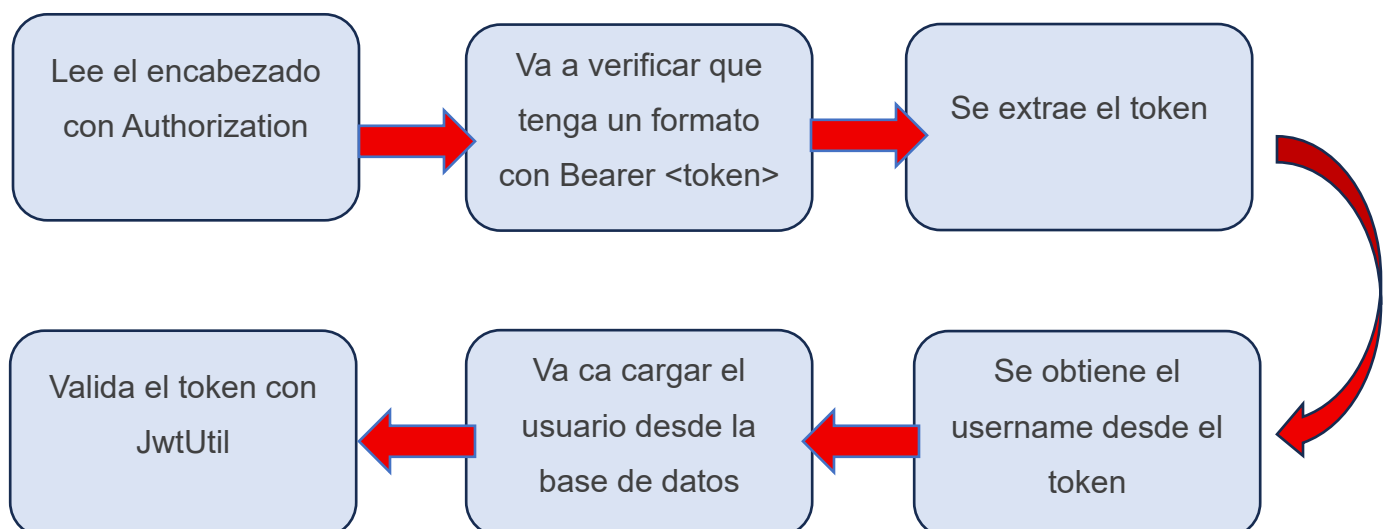
Cuando se recibe un token, JwtUtil valida:

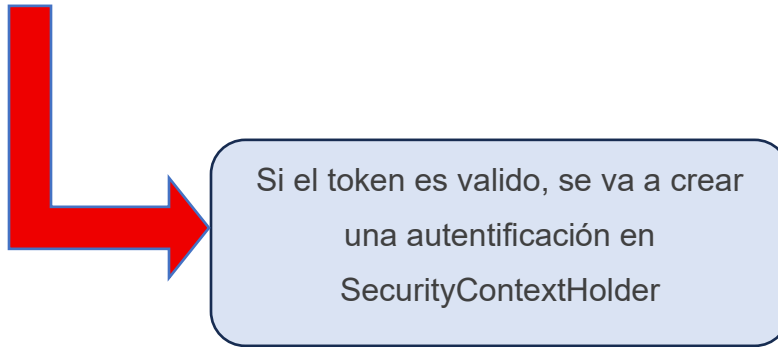
- 1- Que sea correcta la firma
- 2- Que el token no esté expirado
- 3- Que el username del token coincida con el usuario cargado desde la base de datos

Si el token fue alterado, la firma deja de coincidir y el token es rechazado

Filtro JWT

La clase JwtAuthenticationFilter va a interceptar cada solicitud HTTP antes de llegar al controlador. El flujo de lo anterior sería el siguiente





Gracias a este filtro, cada vez que la request se encuentre protegida puede identificar el usuario autenticado y sus roles.

Configuración principal de Sprin Security

En nuestra clase SecurityConfig define el comportamiento de seguridad del backend mediante un SecurityFilterChain. Para ello se utilizaron las siguientes configuraciones principales:

- CSRF deshabilitado para API REST stateless
- Consola H2 permitida para pruebas en local
- Sesiones deshabilitadas con SessionCreationPolicy.STATELESS
- Endpoints públicos definidos
- Endpoints públicos definidos
- Endpoints protegidos según rol
- Filtro JWT registrados antes de UsernamePasswordAuthenticationFilter
- httpBasic, formLogin y logout deshabilitados

Las reglas principales de acceso son las siguientes:

RECURSOS	ROLES PERMITIDOS
GET /api/productos/**	CLIENTE`, `EMPLEADO`, `ADMINISTRADOR

GET /api/categorias/**	CLIENTE`, `EMPLEADO`, `ADMINISTRADOR
/api/carrito/**	CLIENTE`, `EMPLEADO`, `ADMINISTRADOR
/api/ventas/**	`EMPLEADO`, `ADMINISTRADOR
/api/detalle-ventas/**	`EMPLEADO`, `ADMINISTRADOR
/api/inventario/**	`EMPLEADO`, `ADMINISTRADOR
/api/usuarios/**	`ADMINISTRADOR

Uso de @PreAuthorize

Además de las reglas configuradas en SecurityConfig, se agregó en esta oportunidad anotaciones de @PreAuthorize en los controladores. Esto va a permitir reforzar la autorización a nivel de método.

Imagen del SecurityConfig con la anotación @EnableMethodSecurity que permite que Spring Security reconozca @PreAuthorize en otros archivos

```

17 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
18
19 @Configuration
20 @EnableMethodSecurity
21 public class SecurityConfig {
22

```

Los @PreAuthorize se encuentran en los controladores para que los tome el SecurityConfig

Tabla de como actual en nuestro código

Implementación de @PreAuthorize	Como actúa
@PreAuthorize ("hasRole('ADMINISTRADOR')")	Se usa en UsuarioController para permitir que solo administradores gestionen usuarios

@PreAuthorize ("hasRole('EMPLEADO','ADMINISTRADOR')")	Se usa en controladores como inventario, ventas y detalle de ventas, porque son operaciones internas del negocio
@PreAuthorize ("hasRole('CLIENTE','EMPLEADO','ADMINISTRADOR')")	Se usa en operaciones permitidas para todos los usuarios autenticados, como consultar catálogos o utilizar carrito.

Imagen del UsuarioController con @PreAuthorize

```

13  @RestController
14  @RequestMapping("/api/usuarios")
15  @PreAuthorize("hasRole('ADMINISTRADOR')")
16  public class UsuarioController {
17

```

Imagen del VentaController con @PreAuthorize

```

12  @RestController
13  @RequestMapping("/api/ventas")
14  @PreAuthorize("hasAnyRole('EMPLEADO', 'ADMINISTRADOR')")
15  public class VentaController {
16

```

Imagen del InventarioController con @PreAuthorize

```

@RestController
@RequestMapping("/api/inventario")
@PreAuthorize("hasAnyRole('EMPLEADO', 'ADMINISTRADOR')")
public class InventarioController {

```

Imagen del DetalleVentaController con @PreAuthorize

```
@RestController
@RequestMapping("/api/detalle-ventas")
@PreAuthorize("hasAnyRole('EMPLEADO', 'ADMINISTRADOR')")
public class DetalleVentaController {
```

Imagen del CarritoController con @PreAuthorize

```
@RestController
@RequestMapping("/api/carrito")
@PreAuthorize("hasAnyRole('CLIENTE', 'EMPLEADO', 'ADMINISTRADOR')")
public class CarritoController {
```

Con esta implementación se cumple lo solicitado para la actividad de esta semana, ya que aplica autorización basada en roles usando Spring Security y anotaciones de seguridad.

Endpoints de autenticación

La clase AuthController va a exponer dos endpoints principales

1- Registro

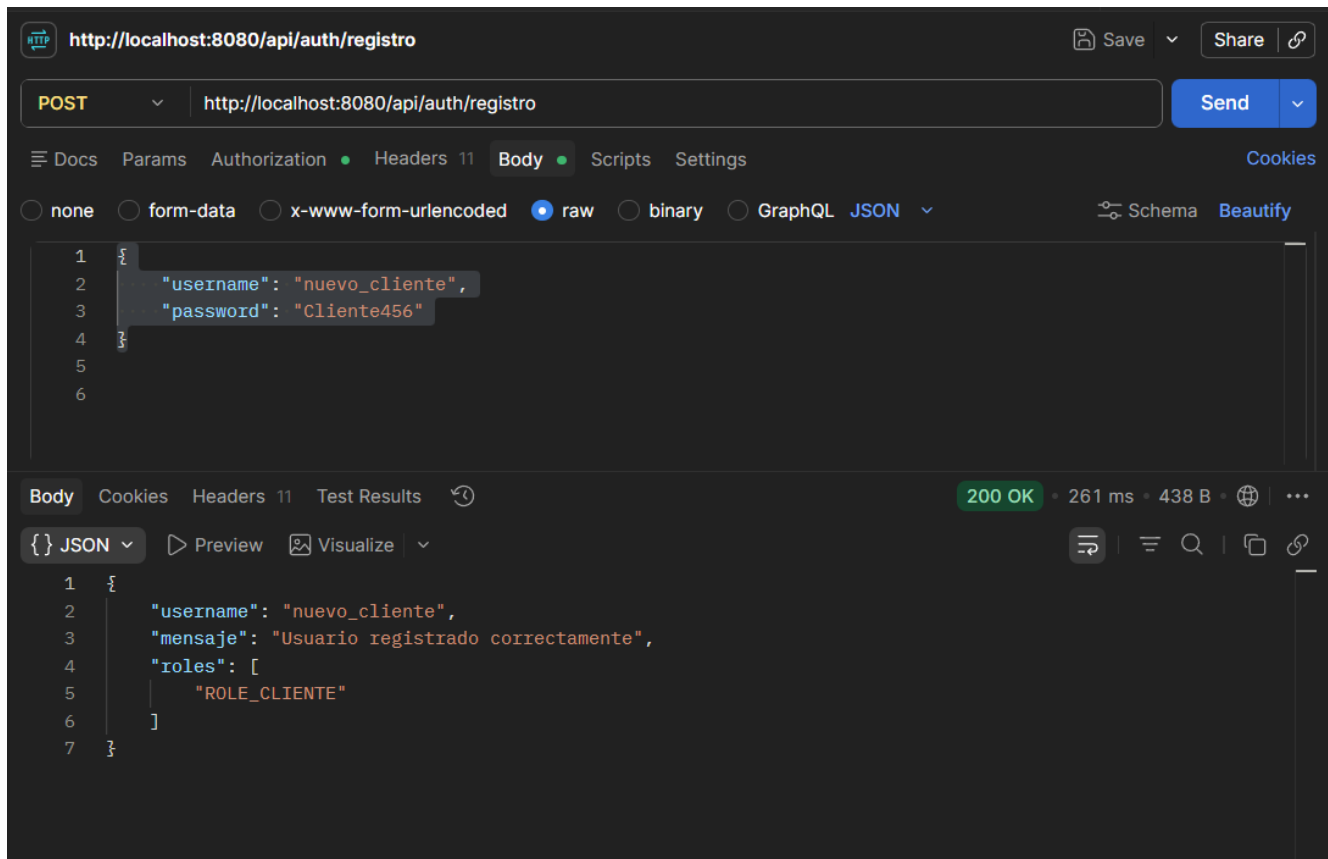
Va a permitir registrar un usuario de tipo cliente. El registro es público siempre asigna el rol de CLIENTE, lo que evita que un usuario se registre a si mismo como empleado o administrador.

Ahora si quisiéramos probar el registro de un nuevo cliente debemos hacer lo siguiente en POSTMAN. Seleccionar el método POST y escribir en la URL lo siguiente, <http://localhost:8080/api/auth/registro> ahora nos vamos a Body y escribimos el nombre del usuario que vamos a crear y su contraseña, mediante el siguiente código JSON:

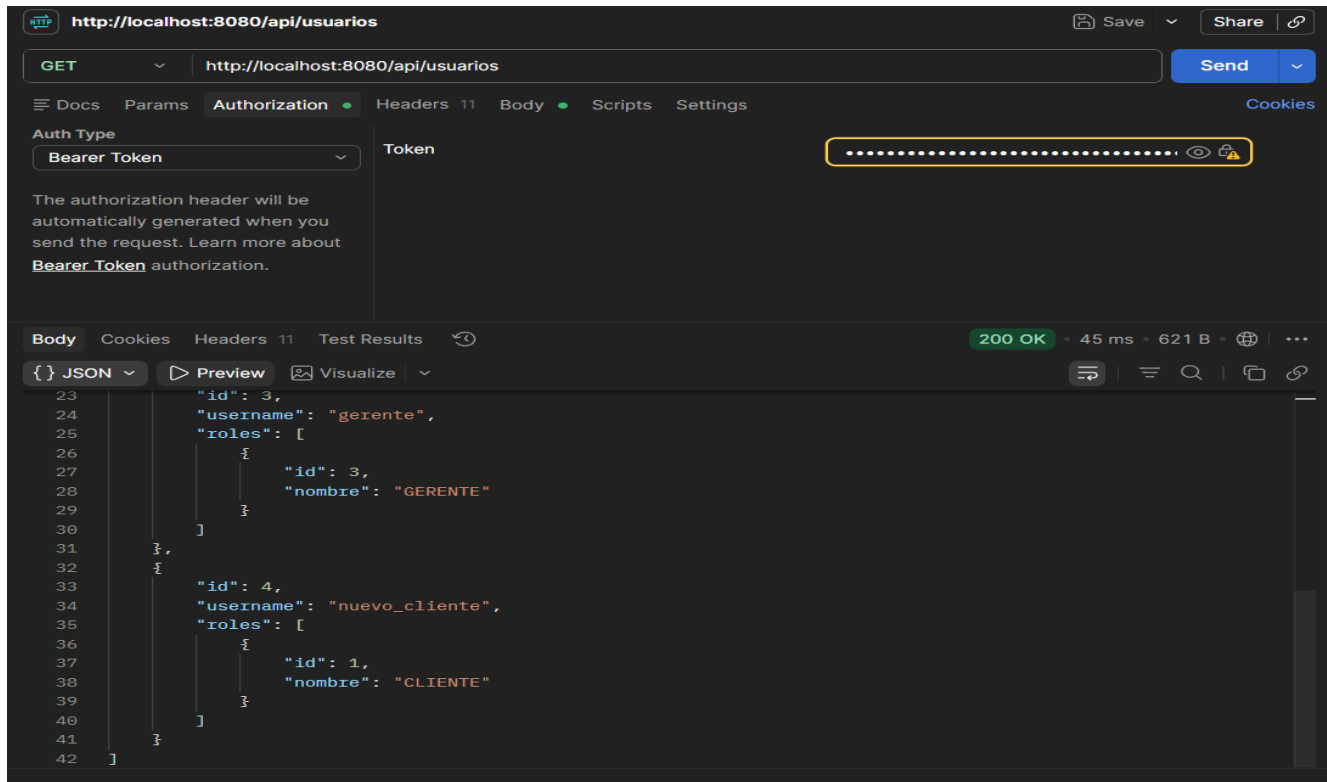
```
{
  "username": "nuevo_cliente",
  "password": "Cliente456"
}
```

Ahora le damos clic al botón SEND y podremos verificar que nuestro usuario se creó de forma correcta, ya que POSTMAN nos debería de devolver un código 200 OK username: "nuevo_cliente", mensaje: "Usuario registrado correctamente" y roles: ROLE_CLIENTE, esto

quiere decir que todos los usuarios que registremos con dicho método quedarán con el rol de cliente.



Ahora si volvemos a hacer login con el usuario gerente y llamamos a todos los usuarios, podremos ver que el nuevo_cliente se creó de forma correcta con el rol de cliente



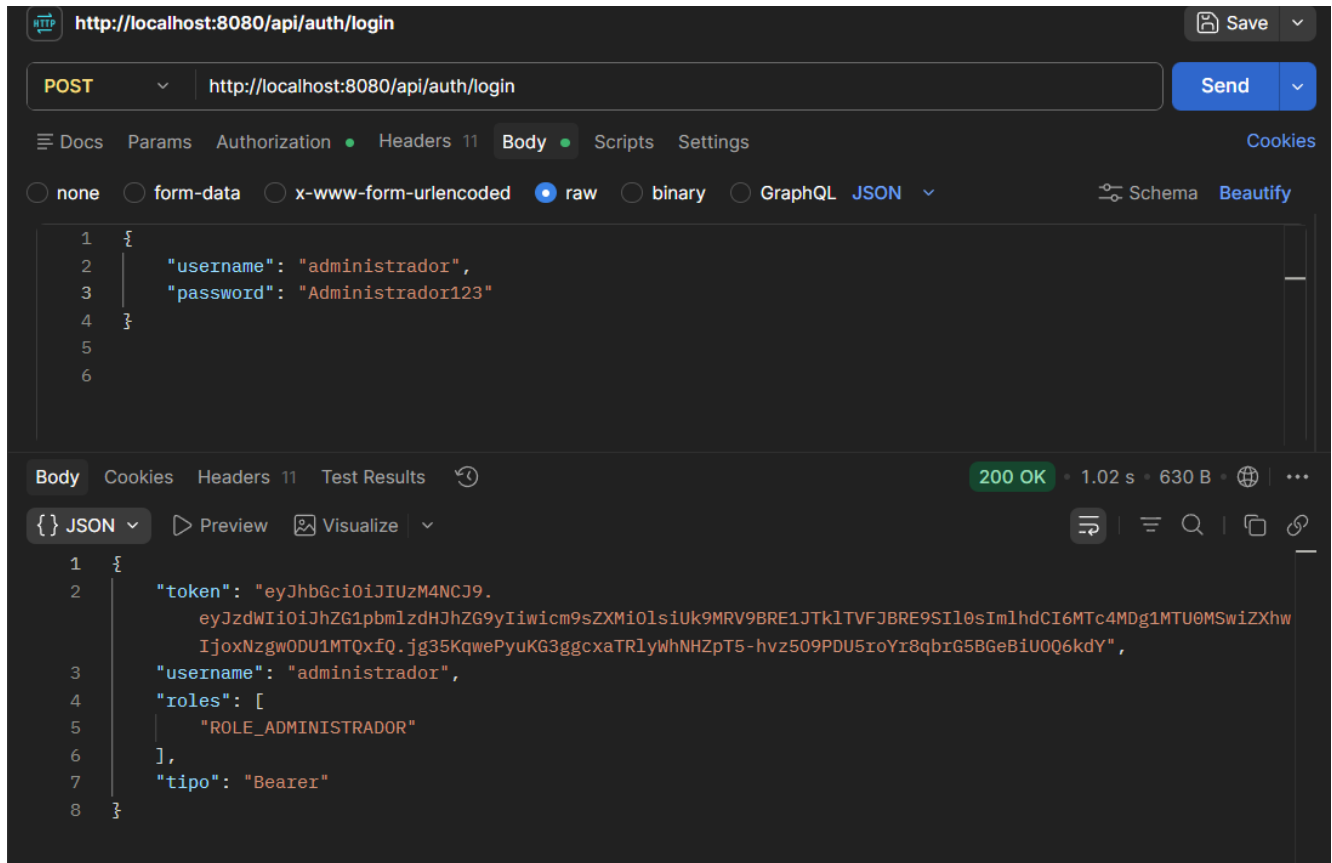
2- Login

El login va a validar las credenciales del usuario y va a devolver un token JWT. Para realizar las pruebas vamos a ejecutar nuestra aplicación y vamos a ingresar en POSTMAN, una vez dentro, vamos a seleccionar el método POST y vamos a escribir en la URL

<http://localhost:8080/api/auth/login> y vamos a escribir el siguiente código JSON:

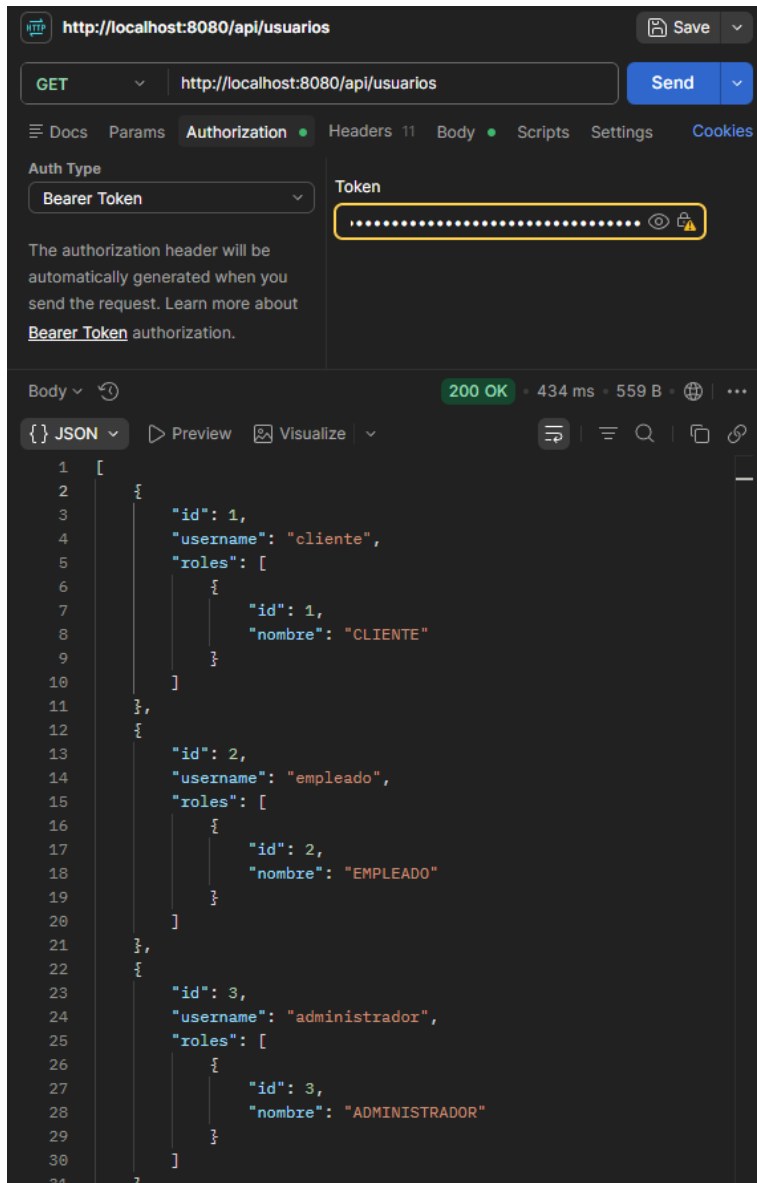
```
{
  username: "administrador",
  password: "Administrador123"
}
```

Al apretar el botón SEND Postman nos debería devolver un código 200 OK y un token. Este token debemos de copiarlo (Sin las comillas)



Uso del token en solicitudes protegidas

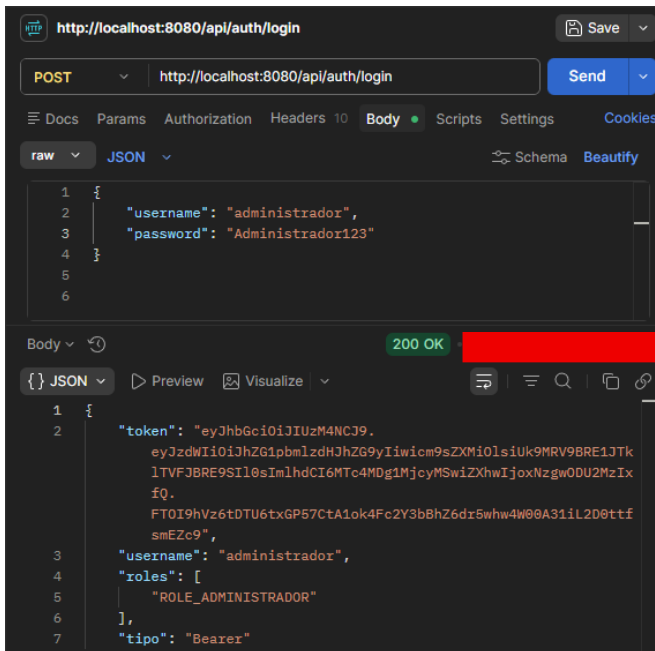
Una vez obtenido el token, el Administrador debe enviarlo en cada solicitud protegida usando el encabezado `Authorization`, para ello nos dirigimos donde dice `Authorization` dentro de POSTMAN y seleccionamos en `Auth Type` donde dice `Bearer Token`. Ahora nos pedirá que ingresemos el token que nos dio en el paso anterior.



Monitoreo básico de autenticación

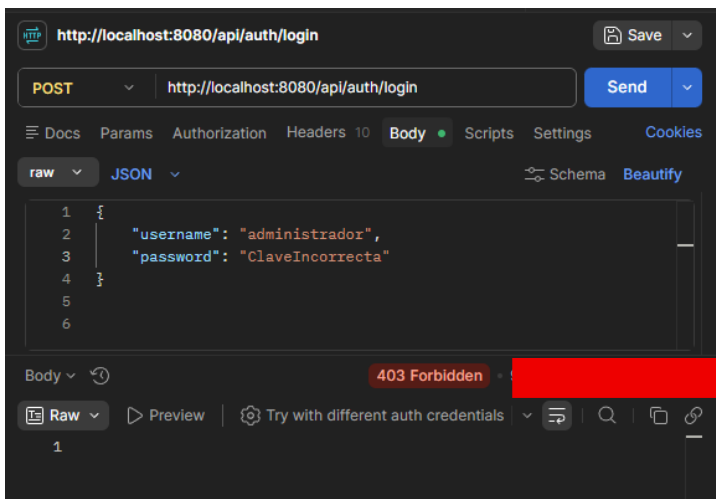
La clase `SecurityAuditService` es una clase que permite auditar los ingresos a mi aplicación y va a registrar los eventos de autenticación como Login Exitoso o Fallido. Esto se ve reflejado en la terminal de nuestro IDE cuando intentamos hacer la ejecución de un ingreso

Ejemplo de Login Exitoso



```
2026-06-07T13:18:41.945-04:00 INFO 34788 --- [minimarket] [nio-8080-exec-5] c.m.s.service.SecurityAuditService : Login exitoso para usuario: administrador
```

Ejemplo de Login Fallido



```
2026-06-07T13:20:49.445-04:00 WARN 34788 --- [minimarket] [nio-8080-exec-8] c.m.s.service.SecurityAuditService : Intento de login fallido para usuario: administrador
```

Resumen de las implementaciones realizadas

Las implementaciones de seguridad de nuestro proyecto van a quedar organizadas de la siguiente forma:

Componente	Responsabilidades
------------	-------------------

SecurityConfig`	Define reglas generales de seguridad, endpoints protegidos y politica stateless
JwtUtil	Genera, firma y valida tokens JWT.
JwtAuthenticationFilter	Intercepta solicitudes y autentica usuarios mediante JWT.
CustomUserDetailsService	Carga usuarios desde la base de datos.
CustomUserDetails	Adapta usuarios y roles al modelo de Spring Security.
AuthController	Expone registro y login.
DataInitializer	Crea roles y usuarios iniciales.
SecurityAuditService	Registra eventos de login exitoso y fallido
Controladores REST	Aplican `@PreAuthorize` segun el rol requerido.

La configuración implementada permite que la aplicación cuente con una autenticación segura basada en JWT, Contraseñas protegidas con BCrypt, sesiones stateless y autorización por roles.

El uso en conjunto de SecurityConfig y @PreAuthorize permite proteher tanto el acceso HTTP general como las operaciones específicas de cada controlador.

3. Explicación técnica de cómo las configuraciones implementadas protegen los componentes backend frente a las amenazas identificadas.

En este punto se explicará técnicamente como las configuraciones de seguridad que se implementaron en el backend protegen los componentes del sistema frente a las amenazas identificadas.

Las soluciones se basan en:

- Spring Security

- Autenticación con usuarios y contraseñas
- Cifrado de contraseñas con BCrypts
- Token JWT firmados
- Sesiones stateless
- Autorización basada en roles
- Anotaciones @PreAuthorize
- Registro básico de eventos de autenticación

Componentes del backend protegidos

Los componentes principales que fueron protegidos son:

- Controladores REST.
- Endpoints de usuarios.
- Endpoints de productos y categorías.
- Endpoints de inventario.
- Endpoints de ventas y detalle de ventas.
- Endpoints de carrito.
- Credenciales de usuarios.
- Roles y permisos.
- Solicitudes HTTP autenticadas mediante JWT.

Protección frente a acceso no autorizado

Amenaza

Un usuario no autenticado o con un rol insuficiente podría intentar acceder a recursos internos del backend, como administración de usuarios, inventario o ventas.

Configuraciones implementadas

En SecurityConfig se definen reglas de acceso por endpoint. Como por ejemplo:

- /api/usuarios/**: solo `ADMINISTRADOR`.
- /api/inventario/**: `EMPLEADO` o `ADMINISTRADOR`.

- /api/ventas/**: `EMPLEADO` o `ADMINISTRADOR`.
- /api/carrito/**: `CLIENTE`, `EMPLEADO` o `ADMINISTRADOR`.
- GET /api/productos/**: `CLIENTE`, `EMPLEADO` o `ADMINISTRADOR`.

Además, se agregaron anotaciones `@PreAuthorize` en los controladores para reforzar la seguridad a nivel de método.

Ejemplo:

```
@PreAuthorize("hasRole('ADMINISTRADOR')")
```

¿Como lo protege?

Spring Security va a validar la identidad y los roles del usuario antes de permitir la ejecución del controlador. Si un usuario no tiene el rol requerido, la solicitud es bloqueada y no llega a ejecutar la lógica de negocio.

Esto protege operaciones críticas como:

- Crear, modificar o eliminar usuarios (CRUD).
- Administrar inventario.
- Registrar ventas.
- Consultar información interna del sistema.

Protección frente a robo o exposición de credenciales

Amenaza

Si las contraseñas se almacenan en texto plano o se exponen en respuestas JSON, un atacante podría obtener credenciales válidas y acceder al sistema.

Configuraciones implementadas

Se configuró BCrypt como mecanismo de cifrado:

```
@Bean
```

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}
```

```
}
```

Las contraseñas se cifran al:

- Crear usuarios iniciales en `DataInitializer`.
- Registrar usuarios en `AuthController`.
- Guardar usuarios en `UsuarioServiceImpl`.

Además, la propiedad password de la entidad Usuario se marca como WRITE_ONLY, evitando que sea enviada en respuestas JSON.

¿Como lo va a proteger?

BCrypt va a transformar la contraseña original en un hash seguro. Aunque alguien acceda a la base de datos, no obtiene la contraseña real del usuario.

El uso de WRITE_ONLY evita que el backend devuelva contraseñas o hashes en respuestas HTTP, reduciendo el riesgo de exposición accidental.

Protección frente a manipulación de tokens

Amenaza

Un atacante podría intentar modificar el contenido de un JWT para cambiar su usuario o agregar roles superiores, como `ADMINISTRADOR`.

Configuraciones implementadas

La clase JwtUtil genera tokens firmados con una clave secreta. El token incluye:

- Username como subject.
- Roles como claim.
- Fecha de emision.
- Fecha de expiracion.
- Firma HMAC.

Durante cada solicitud, el token se valida usando la misma clave secreta.

¿Como lo protege?

Si un atacante modifica cualquier parte del token, la firma deja de coincidir. En ese caso, JwtUtil rechaza el token y el filtro JWT no autentica al usuario.

Esto evita que un usuario altere manualmente sus permisos o suplante otra identidad mediante un token modificado.

Protección frente a sesiones inseguras

Amenaza

Las sesiones tradicionales guardadas en el servidor pueden generar dependencia del estado interno de la aplicación. En arquitecturas distribuidas o de microservicios, esto dificulta la escalabilidad y puede provocar problemas si una instancia no conoce la sesión creada por otra.

Configuraciones implementadas

En SecurityConfig se utiliza:

SessionCreationPolicy.STATELESS

También se deshabilitan mecanismos tradicionales:

- formLogin
- httpBasic
- logout

¿Cómo lo protege?

El backend no guarda sesiones en memoria. Cada solicitud debe enviar su JWT, y el servidor lo valida de forma independiente.

Esto mejora la seguridad y escalabilidad porque:

- No existe una sesion del lado servidor que mantener.
- Cada request se valida por separado.
- La aplicacion puede escalar a multiples instancias con menor dependencia interna.

Protección frente a CSRF

Amenaza

CSRF ocurre cuando un atacante induce al navegador de un usuario autenticado a ejecutar una acción no deseada usando cookies de sesión.

Configuraciones implementadas

En SecurityConfig se deshabilita CSRF:

```
.csrf(csrf -> csrf.disable())
```

Esto se acompaña de una arquitectura stateless basada en JWT enviado en el encabezado:

Authorization: Bearer TOKEN

¿Como lo protege?

En este backend no se usan cookies de sesión para autenticar automáticamente solicitudes. El cliente debe enviar explícitamente el token JWT en el encabezado Authorization.

Por esta razón, el riesgo de CSRF disminuye respecto a una aplicación web tradicional basada en cookies.

Protección frente a inyecciones SQL

Amenaza

Las inyecciones SQL ocurren cuando datos enviados por el usuario son concatenados directamente en consultas SQL, permitiendo alterar la consulta original.

Configuraciones implementadas

El proyecto utiliza:

- Spring Data JPA.
- Repositorios como UsuarioRepository, ProductoRepository, RolRepository, entre otros.
- Métodos de repositorio como findByUsername.

¿Como lo protege?

Spring Data JPA genera consultas parametrizadas y evita construir SQL mediante concatenación manual. Esto reduce significativamente el riesgo de las inyecciones SQL.

Además, al usar entidades y repositorios, la lógica de acceso a datos queda centralizada y controlada.

Proteccion frente a XSS

Amenaza

XSS puede ocurrir cuando datos ingresados por usuarios son devueltos y luego renderizados por un frontend sin sanitización o limpieza.

Configuraciones implementadas

El backend no renderiza paginas HTML como vista principal; expone una API REST. Además, protege credenciales y datos sensibles, como la contraseña, evitando su exposición en respuestas JSON.

¿Como lo protege?

Al no renderizar HTML directamente, el backend reduce el punto de entrada para XSS del lado servidor. Sin embargo, el riesgo no desaparece por completo, porque un frontend podría mostrar datos recibidos desde la API.

Por eso, se recomienda complementar esta implementación con:

- Validacion de entradas.
- Sanitizacion en el frontend.
- Políticas de contenido seguras si existe una interfaz web.

Protección frente a escalamiento de privilegios

Amenaza

Un cliente podría intentar registrarse o modificar su usuario para obtener permisos de empleado o administrador.

Configuraciones implementadas

El endpoint público de registro en AuthController asigna automaticamente el rol CLIENTE.

Los roles superiores no se asignan desde el registro público.

La administración de usuarios queda restringida a ADMINISTRADOR.

¿Como lo protege?

Un usuario nuevo no puede registrarse como administrador desde la API pública. Para crear o modificar usuarios con roles superiores, se requiere acceder a endpoints protegidos con rol administrativo.

Esto aplica el principio de mínimo privilegio y reduce el riesgo de escalamiento indebido.

Protección frente a solicitudes sin autenticación

Amenaza

Un atacante podría intentar acceder a endpoints protegidos sin enviar token.

Configuraciones implementadas

El JwtAuthenticationFilter va a revisar cada solicitud y busca el encabezado:

Authorization: Bearer <token>

Si no existe token, la solicitud continua sin usuario autenticado. Luego Spring Security evalúa si el endpoint requiere autenticación.

¿Cómo lo protege?

Si el endpoint está protegido, Spring Security rechaza la solicitud porque no existe una autenticación válida en el `SecurityContextHolder`.

Esto evita que usuarios anónimos accedan a recursos internos.

Protección frente a tokens expirados

Amenaza

Si un token no expira, podría ser usado indefinidamente si es robado.

Configuraciones implementadas

El token JWT incluye fecha de expiración configurada mediante:

```
app.jwt.expiration-ms=3600000
```

Esto se realiza dentro del archivo application.properties

Esto equivale a una duración de una hora.

¿Como lo protege?

JwtUtil valida que el token no este expirado antes de autenticar al usuario.

Si el token supera su tiempo de vida, se rechaza y el usuario debe iniciar sesion nuevamente.

Esto limita el periodo de uso de un token comprometido.

Protección mediante monitoreo básico

Amenaza

Los intentos repetidos de login fallido pueden indicar fuerza bruta, credenciales filtradas o actividad sospechosa.

Configuraciones implementadas

La clase SecurityAuditService escucha eventos de autenticación:

- AuthenticationSuccessEvent
- AbstractAuthenticationFailureEvent

Registra en consola:

- Login exitoso.
- Login fallido.

¿Como lo protege?

El registro de eventos permite detectar actividad anómala, como múltiples intentos fallidos de acceso.

Aunque es un monitoreo básico, entrega evidencia útil para revisar comportamiento sospechoso durante pruebas o ejecución local.

Protección por capas

La implementación utiliza seguridad en más de una capa:

Capa	Protección
Configuracion HTTP	Reglas en `SecurityConfig` por ruta y metodo.
Metodo/controlador	Restricciones con `@PreAuthorize`.
Autenticacion	Validacion de credenciales con `AuthenticationManager`.
Token	Firma y expiracion mediante `JwtUtil`.
Persistencia	Usuarios y roles cargados desde base de datos.
Credenciales	Hash BCrypt y ocultamiento de password en JSON.
Auditoria	Logs de login exitoso y fallido.

Esta defensa por capas reduce el riesgo de que una única falla exponga completamente el backend.

Conclusión de la implementación

Las configuraciones implementadas protegen el backend de MiniMarket Plus porque controlan quien puede autenticarse, que recursos puede utilizar cada rol y como se validan las solicitudes protegidas.

Spring Security bloquea accesos no autorizados, JWT permite trabajar sin sesiones del servidor, BCrypt protege las credenciales, `@PreAuthorize` refuerza permisos en controladores y `SecurityAuditService` entrega monitoreo basico de eventos de autenticación.

En conjunto, estas medidas mitigan amenazas como acceso no autorizado, exposición de credenciales, manipulación de tokens, uso indebido de sesiones, inyección SQL, CSRF y escalamiento de privilegios.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.